MovieMind: A Hybrid Movie Recommendation System on MovieLens 1M

Mahaboob Ali Ashraf Mohammad
IIT Delhi — AIML Capstone Project

June 21, 2026

Contents

1 Abstract

Movie recommendation systems must predict user preferences from sparse, long-tailed interaction data. This project presents **MovieMind**, an end-to-end pipeline on the MovieLens 1M dataset that combines exploratory analysis, leakage-safe feature engineering, classical machine learning, and neural collaborative filtering (NCF). We evaluate models with root mean squared error (RMSE) and mean absolute error (MAE) on a chronological 70/10/20 train/validation/test split. Among all families tested, **raw Gradient Boosting** on engineered tabular features achieves the best test performance (RMSE **0.8981**, MAE **0.7062**). Hyperparameter tuning improves the deep learning path but does not surpass the tree-based champion. The full workflow is reproduced in `notebooks/MovieMind_capstone.ipynb`; an optional FastAPI and Streamlit deployment extends the work beyond offline modeling.

Keywords: recommender systems, MovieLens, gradient boosting, neural collaborative filtering, cold start, long tail.

2 Introduction

Online streaming platforms depend on recommender systems to surface relevant items from catalogs that users will never fully browse. The problem is difficult because user–item interaction matrices are extremely sparse: most users rate only a small fraction of available movies, and popularity follows a long-tail distribution where niche titles receive few ratings.

MovieLens 1M provides approximately one million explicit star ratings from roughly 6,000 users on about 3,900 movies, along with user demographics and movie metadata [4]. It is a standard benchmark for rating prediction and hybrid recommendation research.

This capstone addresses three practical goals:

1. Understand data characteristics that affect model choice (sparsity, demographics, cold start).
2. Build and compare baseline, machine learning, and deep learning predictors under a strict time-based evaluation protocol.
3. Deliver reproducible artifacts: a single Jupyter notebook for training and evaluation, plus evidence tables committed to the repository.

Accurate offline evaluation matters because deploying a weak model wastes user attention, while overfitting to random splits inflates reported metrics. We therefore split data chronologically by rating timestamp so that training never sees future interactions—a setting closer to production deployment than a random shuffle.

3 Literature Review

3.1 Collaborative filtering and matrix factorization

Early recommender systems relied on **collaborative filtering**: inferring preferences from patterns in the user–item matrix. Matrix factorization and latent-factor models remain strong baselines when interaction density is moderate [8].

3.2 Hybrid and content-aware approaches

Hybrid systems combine collaborative signals with content features (genres, demographics, release year). Engineered aggregates—such as per-user average rating and per-movie popularity—inject side information without requiring dense pairwise co-occurrence. Tree ensembles (random forests, gradient boosting) are effective on such tabular data [2, 3].

3.3 Neural collaborative filtering

He et al. [5] proposed **Neural Collaborative Filtering (NCF)**, which learns user and item embeddings and passes their interaction through a multilayer perceptron to predict ratings. NCF can capture nonlinear interactions but typically needs more data and tuning than strong gradient-boosted baselines on medium-scale explicit-feedback datasets.

3.4 Hyperparameter optimization

Modern DL pipelines often use automated search. **Optuna** provides efficient trial-based optimization for neural architectures and learning rates [1]. We use Optuna for NCF and grid search for sklearn models, selecting configurations on the validation split before reporting test metrics.

3.5 Implementation stack

Modeling uses **scikit-learn** [7] for classical ML and **PyTorch** [6] for NCF. The MovieLens 1M schema and separator conventions follow the official GroupLens release [4].

4 Data Exploration

4.1 Dataset overview

MovieLens 1M contains three core tables:

- **Ratings** (`userId`, `movieId`, `rating`, `timestamp`)
- **Movies** (`movieId`, `title`, `genres`)
- **Users** (`userId`, `gender`, `age`, `occupation`, `zipCode`)

The loaded corpus contains **1,000,209** ratings from **6,040** users on **3,883** catalog movies (3,706 movies with at least one rating in the interaction matrix). Matrix sparsity is **95.74%**—only about 4.3% of possible user–movie pairs are observed—which is typical for explicit-feedback recommenders.

4.2 Rating distribution

Users skew toward positive ratings: mean rating **3.58** stars, median **4** stars, with the highest count at 4 stars (Figure 1). This generosity bias implies that RMSE penalizes under-prediction of high ratings differently than symmetric error would suggest, and that a global-mean baseline is a meaningful floor.

4.3 Demographics

Gender and binned age categories are unevenly distributed (Figure 2). Occupation and age codes are categorical; they are included as numeric features in our ML pipeline for simplicity, though one-hot encoding could be explored in future work.

4.4 Long tail and cold start

Movie popularity is highly skewed: a small head of blockbusters accumulates thousands of ratings, while **663** movies (17.9% of rated titles) receive fewer than 20 ratings—our cold-start threshold (Figure 3). User activity is similarly skewed: **1,743** users (28.9%) submitted fewer than 50 ratings (Figure 4).

These patterns motivate:

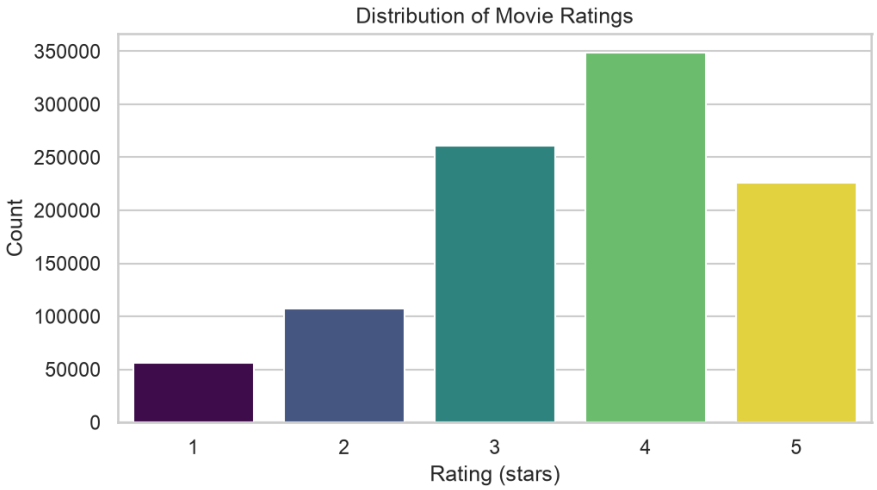

Figure 1: Distribution of explicit star ratings in MovieLens 1M.

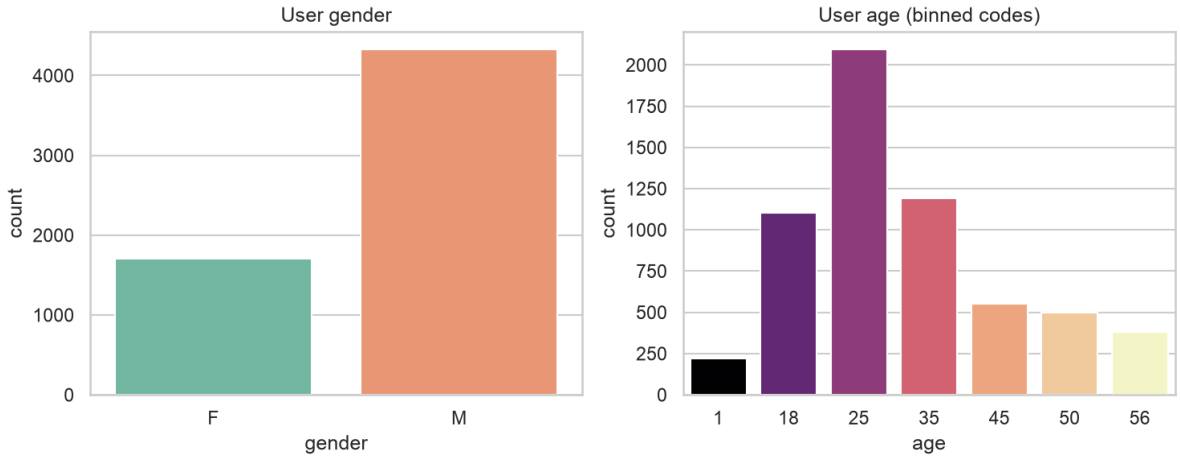

Figure 2: User gender and age-bin distributions.

- Popularity and average-rating features at the movie level.
- Activity and average-rating features at the user level.
- Hybrid models that do not rely solely on pairwise co-occurrence for tail items.

The same plots are reproduced in `notebooks/MovieMind_capstone.ipynb` (Section 2). Regenerate figures with: `python3 docs/report/generate_figures.py` from the project root.

5 Methodology

5.1 Evaluation protocol

We use a **chronological 70/10/20 split** on rating timestamps:

- Train: 700,146 rows (earliest 70% of timeline)
- Validation: 100,020 rows (next 10%)
- Test: 200,043 rows (final 20%)

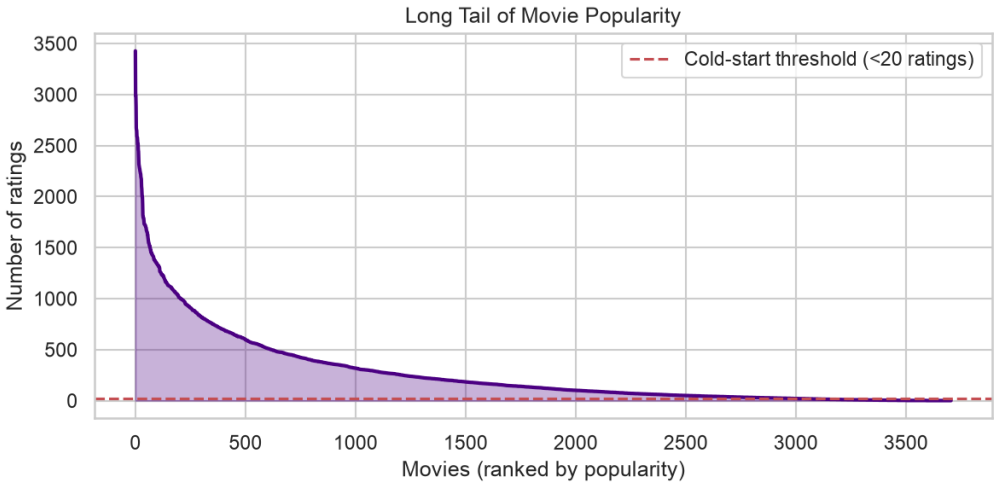

Figure 3: Long-tail distribution of movie popularity (ranked by rating count).

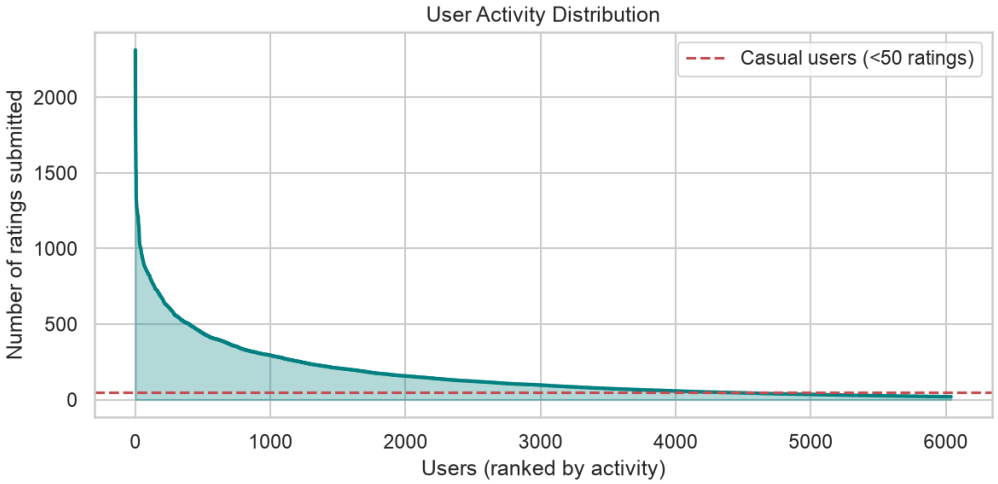

Figure 4: Distribution of user rating activity.

Hyperparameters are selected using validation RMSE; test metrics are reported once per model family. This avoids leakage from future ratings into training features or tuning decisions.

5.2 Feature engineering

From raw tables we compute:

- `user_rating_count`, `user_avg_rating`
- `movie_rating_count`, `movie_avg_rating`
- `release_year` (parsed from movie title)
- User fields: `age`, `occupation`

Implementation: `src/features.py`, orchestrated in the capstone notebook.

5.3 Machine learning models

We train four regressors on the tabular feature matrix:

1. **Baseline:** global mean rating on the training set.
2. **Linear regression** (sklearn defaults).
3. **Random forest** (`n_estimators=100`, `max_depth=10`).
4. **Gradient boosting** (`n_estimators=200`, `max_depth=5`, `learning_rate=0.1`).

ML hyperparameter search (`src/tune_ml.py`) uses a fast-pass subsample of train/validation for grid search, then refits the best configuration on the full training split.

5.4 Neural collaborative filtering

The NCF model (`src/dl_model.py`) embeds users and movies, concatenates embeddings with dense tabular features, and passes them through fully connected layers with dropout. Baseline settings include embedding dimension 32, learning rate 0.001, and 10 training epochs.

Optuna tuning (`src/tune_dl.py`) runs 50 trials with 3 epochs per trial on validation RMSE. The winning configuration is retrained in `src/post_analysis.py` to produce `ncf_tuned_best.pt` and explainability artifacts.

5.5 Metrics

Primary offline metrics:

- **RMSE** — $\sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$; penalizes large errors.
- **MAE** — mean absolute error; more robust to outliers.

Ranking metrics (Precision@K, Recall@K) are defined in `src/evaluation.py` for extension; the capstone notebook focuses on rating regression consistent with the training objective.

5.6 Reproducibility

All steps are runnable from `notebooks/MovieMind_capstone.ipynb`. Reviewers can verify quickly with:

```
bash scripts/verify_capstone_notebook.sh
```

6 Evaluation

6.1 Raw machine learning results (test set)

Table 1 summarizes models trained with default or project-standard hyperparameters.

Table 1: Raw ML and baseline metrics on the test split (70/10/20 protocol).

Model	RMSE	MAE	Fit time (s)
Baseline (global mean)	1.1048	0.9181	0.00
Linear regression	0.9005	0.7088	0.06
Random forest (raw)	0.8998	0.7084	20.72
Gradient boosting (raw)	**0.8981**	**0.7062**	170.06

Gradient boosting achieves the lowest test RMSE and MAE among all families. Linear regression and random forest are competitive, confirming that engineered aggregates capture much of the signal in MovieLens 1M.

6.2 Tuned machine learning

Validation-selected hyperparameters for random forest and gradient boosting yield test RMSE 0.8998 and 0.8993 respectively—slightly *worse* than the raw gradient boosting run (Table B in project evidence). Fast-pass subsampled tuning did not improve the champion; raw GB remains the reporting winner.

6.3 Deep learning results

Table 2: Neural collaborative filtering (validation-focused metrics).

Model	Target	Best / final metric
NCF (raw)	Val RMSE at epoch 10	1.1997
NCF (Optuna tuned)	Val RMSE (50 trials)	1.0887

Tuned NCF substantially improves over the raw architecture on validation RMSE but still trails gradient boosting on the primary test regression metrics used for ML comparison.

6.4 Champion selection

- **Overall offline champion:** raw Gradient Boosting (test RMSE 0.8981).
- **Best DL path:** Optuna-tuned NCF on validation objective.
- **Trade-off:** GB training takes $\sim$170 s vs. $<$1 s for linear regression; NCF tuning adds $\sim$16 minutes of trial search in our recorded run.

Full tables and run logs: `evidence/phase9_split_eval/evaluation_master_summary_70_10_20.md`.

7 Conclusion

We built MovieMind, a reproducible movie recommendation pipeline on MovieLens 1M with explicit attention to data sparsity, long-tail popularity, and leakage-safe evaluation. Engineered tabular features combined with gradient boosting outperform linear models, random forests, and our NCF implementations on test RMSE under a chronological 70/10/20 split.

Key findings:

- More than 95% matrix sparsity and long-tail item popularity justify hybrid features beyond pure collaborative filtering.
- Raw gradient boosting is the strongest single offline model (RMSE 0.8981, MAE 0.7062).
- ML and DL hyperparameter tuning improved some configurations but did not displace the raw GB champion in this run.

Limitations:

- Rating prediction RMSE does not fully capture ranking quality for top-K recommendation lists.
- Cold-start users and tail movies remain challenging despite aggregate features.
- DL models may benefit from longer training and richer text or genre embeddings.

Future work:

- Two-tower retrieval and learning-to-rank objectives (Precision@K / NDCG).
- Session-aware or sequential recommenders using timestamps.
- Integration with the deployed FastAPI/Streamlit app and optional Ollama tool agent for interactive queries (documented separately in the repository).

The complete training and evaluation workflow is available in `notebooks/MovieMind_capstone.ipynb` on the project repository.

8 References

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019.

[2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[3] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[4] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015.

[5] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*, pages 173–182, 2017.

[6] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[8] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence*, pages 452–461, 2009.